HStore and HyperGraphDB: a storage-level benchmark of two persistent hypergraph engines

Metacog Labs

HStore 0.1.0, commit `459b8be`

October 2026

Abstract

We compare HStore, a persistent hypergraph storage engine built on copy-on-write counted B+trees, with HyperGraphDB 1.4 on Berkeley DB Java Edition. Both systems run 26 storage-level workloads on the same skewed synthetic hypergraph, with up to 1.45 million incidences and equal 64 MiB caches, in three configurations: two data sizes and two commit modes. Every read result is cross-checked between the systems. At scale 1 with asynchronous commits, HStore enumerates incidence sets 10.1× faster, counts shared hyperedges 13.8× faster, applies batched updates 15.3× faster, and needs 4.74× less disk. These leads come from storing incidence sets as small sorted trees, and from commits that log only the keys they changed while a background checkpoint writes pages. When the data is four times larger than the cache, HStore keeps most of these leads but falls to 0.13× of HyperGraphDB's speed on member enumeration, because its member lookups miss the node cache. With synchronous commits, both systems are bound by the cost of one forced write per commit. We also report latency percentiles, bytes written, heap use, space reclamation and crash recovery, the variation over runs, and a durability setting in HyperGraphDB that, left unchecked, makes its synchronous commits unsynced.

1 Introduction

Many datasets record facts that involve more than two things at once. An insurance claim joins a patient, a provider, a drug and a pharmacy. A paper joins its authors, and a meeting joins its participants. A hypergraph stores each such fact as one hyperedge over all of its participants. A property graph has to break the same fact into a new node and one edge per participant, and a relational schema into a join table. Queries then ask hypergraph questions: which facts involve this entity, which facts involve both of these entities, and what lies two hops away.

HyperGraphDB [1] is the established open-source hypergraph database for the JVM. It stores atoms and links in Berkeley DB Java Edition (JE), a general-purpose log-structured key-value store. HStore is a newer engine built for this data model. Its incidence sets and hyperedge memberships are counted B+trees, and its storage, transaction and recovery design is described in section 2.

This report compares the two systems at the storage level, on the same machine and data, with the same cache budget. We measure what an embedded application sees: throughput of loads, updates, deletes and reads; per-operation latency, including tail latency while another thread writes; behaviour of giant hyperedges; disk footprint before and after deletes and space reclamation; time to recover after a crash; and the bytes written, heap allocated and GC time behind each result. Every read result is cross-checked between the two systems.

Section 3 describes the dataset, workloads and measurement method. Section 4 presents the results, and section 5 explains them in terms of each system's design. Section 6 lists the limits of this study. Full tables, per-run values and the steps to reproduce every number are in the appendices.

Summary of findings.

- **Incidence-based reads** are faster in HStore in every configuration: 2.64× to 10.1× for incidence sets, 8.62× to 29.0× for co-membership counts, and 29.2× to 65.9× directly after a reopen.
- **Writes** cost HStore far fewer bytes. Single-update commits write 649 B against 9.1 KiB, and batched updates run 11.2× to 16.8× faster. Bulk insertion is closer: between 1.21× and 6.15× for hyperedges.
- **Member enumeration** is HStore's weakness once the data outgrows its node cache: 0.13× at scale 4, and 0.19× for two-hop neighbourhoods, which depend on it.
- **Tail latency.** HStore's p99 and p99.9 latencies are lower than HyperGraphDB's in every latency workload and configuration, by 1.3 to 69 times. The single slowest commit at scale 4 was HStore's.
- **Disk.** HStore's files are 3.85× to 14.3× smaller after loading. After deletes they first grow while retained history still references the replaced pages.
- **Recovery.** Neither system lost an acknowledged commit after SIGKILL. Recovery times overlap.

2 Systems under test

Both systems are embedded Java libraries. Each runs in its own JVM process, and the benchmark harness calls its API directly. Neither is accessed over a network.

2.1 HStore

Data model. Every stored object is an *atom* with a 64-bit identifier. An atom is a node or a hyperedge. A hyperedge has any number of members, and a member can itself be a hyperedge. Hyperedges are either *set* edges, keyed by member, or *ordered* edges, keyed by position. Each membership carries a role set, a weight, a validity interval and a qualifier. The benchmark uses set edges and plain memberships.

Layout. All persistent state is held in copy-on-write counted B+trees. Every child reference stores a summary of its subtree: the entry count, the key range and an order-independent fingerprint. Three trees matter here:

- The *catalog* maps an atom id to its record. A hyperedge record holds a reference to that edge's *membership tree*, keyed by member id. If the membership tree is one leaf of at most 32 entries that fits in about 100 bytes, it is stored inside the record itself and needs no page of its own.
- The *reverse incidence index* maps an atom id to the set of hyperedges that contain it. Sets of up to 48 entries are stored inline in the index leaf. Larger sets are promoted to a tree of their own.
- Secondary indexes on node keys and properties, which the benchmark touches only through node insertion and updates.

Enumerating an incidence set is a cursor walk over one inline list or one small tree. Counting the hyperedges that contain two given atoms is a leapfrog intersection [3] of their two incidence trees. The intersection skips whole subtrees whose key range cannot contain a match.

Pages and files. A tree node is written as a self-describing image: an 80-byte header with a CRC32C checksum, followed by the payload. Images are packed into append-only segment files of 256 MiB, aligned to 64-byte units. A page directory maps each page number to the image's current address, so compaction can move an image without rewriting the trees that refer to it. Reads go through a memory map of each segment. Decoded nodes are kept in a node cache whose budget is set in bytes.

Generations and readers. A committed state is a *generation*: an immutable vector of tree roots. A reader pins a generation and reads it without locks. Writers never modify a node that a published generation can reach, as in other copy-on-write B+tree stores [4]. The last 64 generations are retained for time travel.

Commits and checkpoints. Figure 1 shows the write path. A transaction collects its changes in a private workspace. At commit it applies them to new copies of the affected tree paths, appends one write-ahead log record listing the keys it changed in each tree, and publishes the new generation. The new tree nodes stay in memory, reachable from the published roots. A background checkpoint later writes every node that no earlier checkpoint wrote, syncs the files, updates page reference counts, and publishes a catalog image. It then swaps the stored references into the current generation and truncates the log. Only the swap holds the commit lock. Memory held by unwritten nodes is capped at 64 MiB. A commit that finds the cap reached waits for a checkpoint.

Two exceptions write pages before the checkpoint. A large bulk load spills finished subtrees to pages while it runs, and a tree that changed in more than 4,096 places is written to pages at commit and logged by reference. Under *synchronous* durability, a commit returns after its log record is fsynced; concurrent commits share one fsync (group commit). Under *asynchronous* durability the fsync is skipped, but commits are still published strictly in order.

Recovery. On open, HStore loads the newest catalog image and replays the logged key changes of every commit after it, in commit order. A commit's changes are applied only if its commit record reached the log, so recovery always yields a prefix of the commit order.

Space reclamation. Every page has a reference count, which changes only at checkpoints. A page no generation can reach is released once no pinned reader can see it. A segment whose live bytes fall below half of the bytes written to it is compacted: its live images are copied to the active segment, the page directory is updated, and the old file is deleted once no reader can still be using it. A background maintenance pass, run every 500 ms, compacts one such segment at a time. An explicit compaction request compacts all of them.

2.2 HyperGraphDB

HyperGraphDB [1] is a general hypergraph database for Java. Its atoms are identified by persistent handles (UUIDs). A link stores the ordered array of its target handles. The benchmark uses `HGPlainLink` for hyperedges and `String` atoms for nodes. The incidence set of an atom, the links pointing to it, is maintained by HyperGraphDB in a separate index. The atom cache keeps recently used atoms and incidence sets in memory as Java objects.

We use the Berkeley DB Java Edition storage implementation (`bdb-je`), the default for HyperGraphDB 1.4. JE is a log-structured B+tree store [2, 6]: every update appends a new version of the changed records and internal nodes to the log. A background cleaner copies the live entries out of underused log files and deletes them. A checkpointer bounds recovery time. Incidence sets are stored as sorted duplicate sets in a JE database. A lookup opens a cursor at the atom's handle.

2.3 Synchronous commits in HyperGraphDB

HyperGraphDB 1.4 begins every JE transaction with an explicit `WRITE_NO_SYNC` durability. That setting overrides the environment-wide `COMMIT_SYNC`, so configuring the environment alone leaves commits unsynced: in a first run of the synchronous configuration, HyperGraphDB committed single updates in about 0.1 ms, while one forced write on

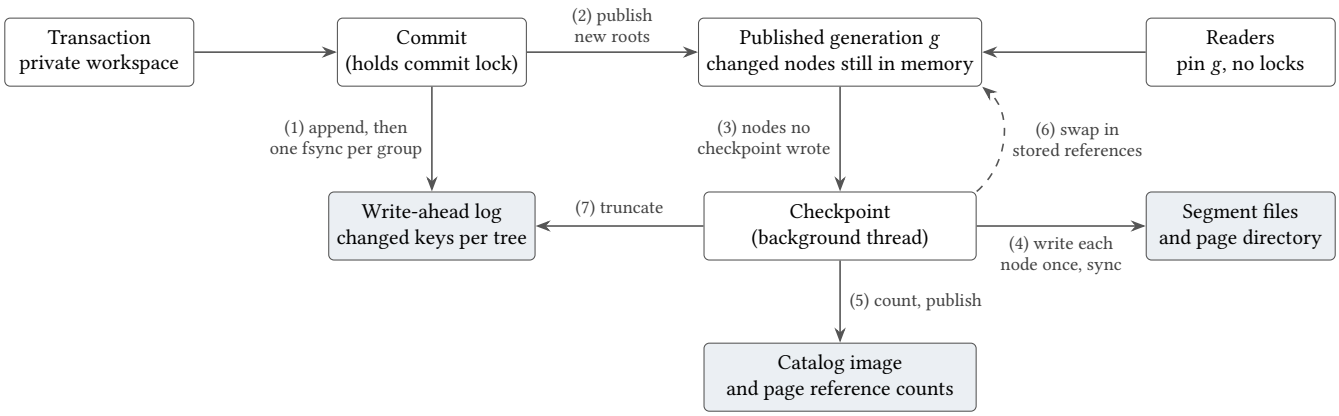


Figure 1: HStore's write path. (1, 2) A commit logs only the keys it changed and publishes new roots whose changed nodes are still in memory. (3–7) A background checkpoint writes those nodes to pages, once for all the commits since the previous checkpoint, then swaps the stored references into the current generation and truncates the log. Only step 6 takes the commit lock.

this machine takes about 3.8 ms. For the synchronous configuration the HyperGraphDB adapter therefore calls `Environment.flushLog(true)` after every commit. This writes and forces JE's log, which is what `COMMIT_SYNC` would do at commit time. The asynchronous configuration is unaffected.

2.4 Versions and configuration

Table 1 lists the configuration of both systems. The two caches are given the same 64 MiB budget. HStore's budget covers decoded nodes. JE's budget covers its B+tree nodes and records, and HyperGraphDB's atom cache comes on top of it. That cache is sized by HyperGraphDB itself and cannot be capped in bytes. The retained-heap figure in section 4.7 shows what each store actually kept.

3 Benchmark design

3.1 Dataset

The dataset is generated from a fixed seed (20260904), so both systems load exactly the same graph. At scale s it has 50,000 s nodes and 100,000 s hyperedges. The cardinality of a hyperedge follows a geometric tail: it starts at 2 and each further member is added with probability 0.62, up to 32, for a mean of 3.6. Members are drawn as $\lfloor n\,u^{2.2} \rfloor$ for uniform $u \in [0, 1)$ over n nodes. Low-numbered nodes therefore become hubs with large incidence sets, as in co-authorship, claims or citation data. Hyperedges never repeat a member.

Table 2 gives the sizes and the skew. The largest incidence set at scale 1 holds 2,645 hyperedges, while half of all nodes are in 5 or fewer. Only 370 incidence sets exceed the 48 entries that HStore stores inline. Probe sets drawn from the same seed hold 100,000 random nodes and hyperedges, 20,000 node pairs taken from a common hyperedge (so that co-membership counts are non-zero), and 20,000 node updates. A separate churn set, drawn after everything else, selects 20% of the hyperedges for deletion and 20,000 other hyperedges, each with at least three members, that lose one member.

3.2 Workloads

Table 3 defines the workloads. They run in the order listed, in one process per system and run, against one database directory. The table also names the API call each system uses. Each adapter uses the plain storage calls of its system and its own identifier type: `long` ids for HStore and persistent handles for HyperGraphDB. Higher-level traversal frameworks, such as HyperGraphDB's breadth-first traversal, are not used, so the numbers reflect the storage layer.

HyperGraphDB links are immutable, so `churn.remove` replaces each link with a new one that lacks the removed member. `read.twohop` runs before `large.ingest`, because once a hyperedge containing every node exists, every node is two hops from all others. `recover` runs in a child JVM with the same flags. The child ingests nodes in transactions of 1,000 into a fresh directory and prints the count after each acknowledged commit. The parent kills it with SIGKILL once half the nodes are acknowledged, then times opening the directory and counts the surviving nodes.

3.3 Measurement

Throughput. Each workload is timed with `System.nanoTime()` around its whole operation set. Throughput is operations divided by elapsed time. Each write workload ends with a flush outside the timed section: an HStore checkpoint, or `Environment.sync()` for JE. Bytes still in buffers are therefore counted against the workload that produced them.

Warm-up. Before every timed read workload except `read.incidence.cold`, the same workload runs untimed for at least 2 s. Both systems get the same warm-up. The numbers describe a process whose JIT has compiled the read path. A short warm-up mostly measures how fast each system's code gets compiled: in our trials, a warm-up over 10% of the

Table 1: System configuration. Both systems use their defaults except where stated.

	HStore	HyperGraphDB
Version	0.1.0, commit `459b8be`, storage format 7	1.4, commit `99485a1`; JE 5.0.73
Cache	node cache, 64 MiB	JE cache 64 MiB, plus atom cache
Async commits	no fsync, ordered publication	JE `WRITE_NO_SYNC` (default)
Sync commits	fsync per commit group	JE log forced after each commit (section 2.3)
Page / node size	16 KiB pages	JE defaults
Space reclamation	compact segments below 50% live	cleaner, 50% minimum utilisation
History	64 generations	none

Table 2: Dataset statistics, computed from the generator. An incidence set is the set of hyperedges that contain a node.

	Scale 1	Scale 4
Nodes	50,000	200,000
Hyperedges	100,000	400,000
Incidences	362,757	1,453,427
Hyperedge cardinality, mean / p99 / max	3.63 / 11 / 30	3.63 / 11 / 30
Incidence set size, mean	7.26	7.27
Incidence set size, p50 / p90 / p99	5 / 12 / 41	5 / 13 / 42
Incidence set size, p99.9 / max	145 / 2,645	142 / 5,714
Nodes in no hyperedge	575	2,319
Incidence sets above 48 entries	370	1,521

probes left `read.members` about three times below its steady rate.

Latency. `latency.read` and `latency.commit` time every transaction separately and report p50, p99, p99.9 and the maximum. In `mixed` and `compaction`, the writer runs on a fixed schedule (open loop), and each commit's latency is measured from the time it was *scheduled* to start. A stall therefore shows up in the latency of every commit queued behind it, instead of silently slowing the writer down. The readers in `mixed.read` run in a closed loop and keep a uniform sample of 200,000 latencies each, plus the exact maximum.

Bytes written and read. Both systems count their own I/O. For HStore this is the log bytes plus the page bytes written, including pages moved by compaction. For HyperGraphDB it is JE's sequential plus random write bytes, including the cleaner. Reads count the bytes each system reads from its files: HStore's segment reads, and JE's sequential plus random read bytes. A read served from the system's own cache counts as zero. A read that reaches a file counts even when the operating system's page cache serves it. Write amplification divides bytes written by an estimate of the logical bytes: 8 bytes per id plus the value, 24 bytes per updated value, $(c+1)\times 8$ bytes for a new or deleted hyperedge of cardinality c, and 16 bytes per removed member.

Memory. For every workload the harness records the bytes allocated, from `ThreadMXBean` (the whole process for write workloads, the reading thread alone for single-threaded reads), and the stop-the-world GC time from `GarbageCollectorMXBean`. After ingest it forces a full GC and records the heap in use, minus a baseline taken before the store opened. This is the *retained heap*.

Correctness cross-check. Every read and delete workload returns a checksum: the number of ids visited, intersected or found. Both systems must return the same checksum in every run, or the result is flagged. All checksums agreed in every run reported here. Workloads whose result depends on timing (`mixed`, `compaction`, `recover`) are not checked.

3.4 Environment and statistics

All runs used one machine: an Apple M5 with 4 performance and 6 efficiency cores, 24 GiB of memory, and a 1 TB Apple SSD formatted with APFS, running macOS 26.5.1. The JVM was Oracle GraalVM for JDK 25.0.4 with the HotSpot JIT, a fixed 4 GiB heap (`-Xms4g -Xmx4g`) and the Parallel collector. Each system ran in its own JVM, and runs alternated between the two systems so that slow periods affect both. We ran three configurations:

- **S1 async**: scale 1, asynchronous commits, 5 runs per system;
- **S1 sync**: scale 1, synchronous commits, 5 runs per system;
- **S4 async**: scale 4, asynchronous commits, 3 runs per system. With 64 MiB caches, the working set is several times larger than either cache.

Each table reports the median over runs. Ratios are taken between medians and oriented so that values above 1 favour HStore: throughput divides HStore by HyperGraphDB, while times, latencies and sizes divide HyperGraphDB by HStore. Where the ranges of the two systems' runs overlap, the ratio is marked with †, and the difference should not be read as significant. Appendix B gives every run and the coefficient of variation of each workload.

4 Results

Figure 2 shows HStore's advantage on every workload in all three configurations, and table 4 gives the same ratios with the run-range test from section 3.4. Table 5 gives the absolute numbers for the S1 async configuration. The other configurations are tabulated in appendix A.

4.1 Loading, updating and deleting

At scale 1 with asynchronous commits, HStore inserts nodes 1.81× and hyperedges 1.25× as fast as HyperGraphDB. Both ratios carry the overlap mark: the slowest HStore runs were

Table 3: Workloads, in execution order. "txn" is a transaction. All read workloads except `read.incidence.cold` run after a warm-up (section 3.3).

Workload	Operation	HStore call	HyperGraphDB call
`ingest.nodes`	insert all nodes, 1,000 per txn	`Writer.node`	`graph.add(String)`
`ingest.edges`	insert all hyperedges, 1,000 per txn	`Writer.edge`, `Writer.load`	`graph.add(HGPlainLink)`
`read.incidence`	enumerate a node's incidence set; 100,000 probes, 1,000 per read txn	`forEachIncidentEdge`	`getIncidenceSet`
`read.members`	enumerate a hyperedge's members; 100,000 probes	`forEachMemberId`	`getTargetAt`
`read.comembership`	count hyperedges containing both nodes of a pair; 20,000 pairs	leapfrog intersection	`hg.and(hg.incident, hg.incident)`
`read.twohop`	count distinct nodes sharing a hyperedge with a node; 10,000 probes	incidence, then members	incidence, then targets
`read.incidence.parallel`	`read.incidence` from 8 threads, 100,000 probes each	as above	as above
`latency.read`	one incidence lookup per read txn, each timed; 100,000 txns	as above	as above
`write.update`	replace a node's value; 20,000 updates, 1,000 per txn	`Writer.set`	`graph.replace`
`latency.commit`	one update per write txn, each timed; 5,000 txns	`Writer.set`	`graph.replace`
`mixed.read`	single incidence lookups from 7 threads for 10 s, beside `mixed.write`	as `latency.read`	as `latency.read`
`mixed.write`	one thread committing 10 updates per txn on a fixed schedule of 1,000 updates/s	`Writer.set`	`graph.replace`
`large.ingest`	create one hyperedge containing every node	`Writer.load`	one `HGPlainLink`
`large.scan`	enumerate the large hyperedge 400 times	`forEachMemberId`	`getTargetAt`
`large.probe`	test whether a node belongs to the large hyperedge; 100,000 probes	membership-tree lookup	`getIncidenceSet(n).contains(e)`
`reopen`	close cleanly and open again	`HypergraphDatabase.open`	`HGEnvironment.get`
`read.incidence.cold`	`read.incidence` immediately after the reopen, no warm-up		
`disk`	bytes on disk after a clean shutdown		
`churn.delete`	delete 20% of the hyperedges, 1,000 per txn	`Writer.delete`	`graph.remove`
`churn.remove`	remove one member from each of 20,000 other hyperedges, 1,000 per txn	`Writer.remove`	`graph.replace` with a new link
`read.incidence.churned`	`read.incidence` after the deletes		
`disk.churned`	bytes on disk after the deletes and a clean shutdown		
`compaction.baseline`	single-update commits at 50/s for 10 s, each timed	`Writer.set`	`graph.replace`
`compaction`	the same commits while a full space reclamation runs, starting 1 s in	`StorageEngine.compact`	`cleanLog` until idle, then checkpoint
`disk.compacted`	bytes on disk after the reclamation and a clean shutdown		
`recover`	open a store whose ingest process was killed with SIGKILL	`HypergraphDatabase.open`	`HGEnvironment.get`

slower than the fastest HyperGraphDB runs, so bulk insertion should be read as roughly even with a lead for HStore. Updates differ more. Replacing node values 1,000 at a time runs 15.3× faster in HStore (78.1 k against 5.1 k updates per second). Deleting hyperedges is 5.46× faster and removing single members 3.00× faster, both with overlapping ranges. Building one hyperedge over all 50,000 nodes takes HStore a single bulk load, 12.3× faster than HyperGraphDB.

Single-update transactions show the smallest write difference: 7.9 k against 5.8 k commits per second (1.36×, ranges overlapping). With asynchronous commits, neither system waits for the disk, and the cost of one commit is dominated by work in memory. Section 4.4 shows that the two medians are close and that the difference lies in the tail.

4.2 Reads

HStore's largest margins are on reads that go through incidence sets. It enumerates a node's incidence set 10.1× faster than HyperGraphDB, counts the hyperedges shared by two nodes 13.8× faster, and computes two-hop neighbourhoods 4.04× faster. With 8 threads, incidence lookups reach 24.44 M per second in HStore against 1.69 M (14.4×). When every lookup is its own read transaction (`latency.read`), the ratio is 10.4×. Enumerating the members of a hyperedge, the read where HyperGraphDB's atom cache helps most, is 2.17× faster in HStore at the median, with overlapping ranges.

Two read workloads run under special conditions. Directly after a reopen, before any warm-up, HStore serves incidence lookups at 2.76 M per second against 41.9 k (65.9×). HStore read 27 B per lookup from its files in that phase, against 5.5 KiB for JE. After the deletes, incidence lookups are 6.22× faster.

4.3 Giant hyperedges

On the hyperedge that contains all 50,000 nodes, HStore scans members 3.70× faster (2.26 G members per second against 610.67 M) and tests membership 20.2× faster. A membership test in HStore is one descent of the edge's member-

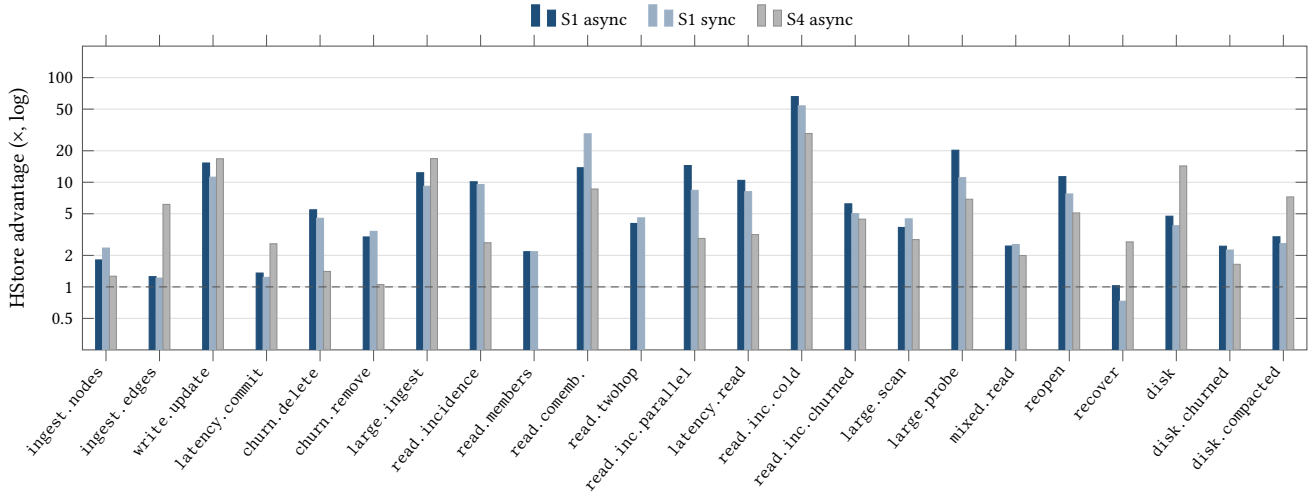


Figure 2: HStore's advantage over HyperGraphDB for every workload and configuration (log scale). Bars above the dashed line favour HStore. Throughput workloads divide HStore by HyperGraphDB; `reopen`, `recover` and the disk sizes divide HyperGraphDB by HStore.

Table 4: Ratio of medians, HStore over HyperGraphDB, oriented so that values above 1 favour HStore (bold). †: the run ranges of the two systems overlap.

Workload	S1 async	S1 sync	S4 async
`ingest.nodes`	**1.81**×†	**2.35**×	**1.27**×†
`ingest.edges`	**1.25**×†	**1.21**×†	**6.15**×
`write.update`	**15.3**×	**11.2**×	**16.8**×
`latency.commit`	**1.36**×†	**1.23**×	**2.58**×†
`churn.delete`	**5.46**×†	**4.53**×	**1.41**×†
`churn.remove`	**3.00**×†	**3.40**×†	**1.05**×†
`large.ingest`	**12.3**×	**9.16**×	**16.8**×
`read.incidence`	**10.1**×	**9.52**×	**2.64**×†
`read.members`	**2.17**×†	**2.17**×†	0.13×
`read.comembership`	**13.8**×	**29.0**×	**8.62**×
`read.twohop`	**4.04**×	**4.57**×	0.19×†
`read.incidence.parallel`	**14.4**×	**8.37**×	**2.90**×
`latency.read`	**10.4**×	**8.13**×	**3.16**×
`read.incidence.cold`	**65.9**×	**53.7**×	**29.2**×
`read.incidence.churned`	**6.22**×	**5.02**×	**4.44**×
`large.scan`	**3.70**×	**4.47**×	**2.83**×
`large.probe`	**20.2**×	**11.1**×	**6.88**×
`mixed.read`	**2.46**×	**2.54**×	**1.99**×
`reopen`	**11.3**×	**7.73**×	**5.09**×
`recover`	**1.03**×†	0.73×†	**2.69**×†
`disk`	**4.74**×	**3.85**×	**14.3**×
`disk.churned`	**2.45**×	**2.25**×	**1.64**×
`disk.compacted`	**3.01**×	**2.59**×	**7.24**×

ship tree.

4.4 Latency

Figure 3 and table 6 give the latency distributions. For single-update commits with asynchronous commits, the medians are close: 85 µs for HStore and 91 µs for HyperGraphDB. The tails are not. At p99, HStore needs 430 µs and HyperGraphDB 1.08 ms. At p99.9 the figures are 1.62 ms and 5.02 ms. Read transactions take 0.3 µs at the median in HStore and 1.6 µs in HyperGraphDB, and 5.2 µs against 207 µs at p99.9.

Reads beside a writer. In `mixed`, the writer's median rate was 1,000 updates per second in both systems; its slowest run reached 996 updates per second in HStore and 795 in HyperGraphDB. Meanwhile the 7 readers completed 1.87 M lookups per second in HStore and 762.0 k in HyperGraphDB (2.46×). Reader latency at p99.9 was 57 µs against 1.03 ms. The writer's commits, measured from their scheduled start, took 16.70 ms against 80.12 ms at p99.

Commits during space reclamation. In the `compaction` window, the median commit latency was about the same with and without a reclamation running, for both systems. At p99, HStore's commits took 9.39 ms during compaction against 9.63 ms without; HyperGraphDB's took 13.32 ms during cleaning against 27.99 ms without.

4.5 Disk footprint, deletes and reclamation

After loading at scale 1, HStore's files take 91.2 MiB and HyperGraphDB's 432.2 MiB (fig. 4). After the deletes the figures are 119.8 MiB and 293.3 MiB. HStore's files grow at this point. The deletes write new pages, and the old ones stay on disk while any of the 64 retained generations can reach them, until a later checkpoint releases them and compaction removes them. After a full reclamation and a clean shutdown, HStore is back to 90.8 MiB, and HyperGraphDB is at 273.5 MiB.

4.6 Reopen and recovery

A clean close and reopen takes 121 ms in HStore and 1,369 ms in HyperGraphDB (11.3×). After the ingest process is killed with SIGKILL, opening the store takes 128 ms and 132 ms respectively. The ranges overlap, so we read recovery time as even. In every run of both systems, every node acknowledged before the kill was present after recovery.

Table 5: S1 async: median throughput (operations per second), times and sizes over 5 runs per system.

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	45.1 k	24.9 k	**1.81**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	28.2 k	22.5 k	**1.25**×	yes
`write.update`	Update node values (1,000 per txn)	78.1 k	5.1 k	**15.3**×	yes
`latency.commit`	Single-update transactions	7.9 k	5.8 k	**1.36**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	16.3 k	3.0 k	**5.46**×	yes
`churn.remove`	Remove one member (1,000 per txn)	24.4 k	8.1 k	**3.00**×	yes
`large.ingest`	Build one giant hyperedge	439.6 k	35.7 k	**12.3**×	yes
`read.incidence`	Incidence set of a node	7.47 M	739.5 k	**10.1**×	yes
`read.members`	Members of a hyperedge	3.69 M	1.70 M	**2.17**×	yes
`read.comembership`	Co-membership count of a pair	447.8 k	32.5 k	**13.8**×	yes
`read.twohop`	Two-hop neighbourhood	301.2 k	74.6 k	**4.04**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	24.44 M	1.69 M	**14.4**×	yes
`latency.read`	One lookup per read txn	2.81 M	269.4 k	**10.4**×	yes
`read.incidence.cold`	Incidence sets after reopen	2.76 M	41.9 k	**65.9**×	yes
`read.incidence.churned`	Incidence sets after deletes	6.89 M	1.11 M	**6.22**×	yes
`large.scan`	Scan the giant hyperedge	2.26 G	610.67 M	**3.70**×	yes
`large.probe`	Membership probe, giant hyperedge	15.62 M	771.9 k	**20.2**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	1.87 M	762.0 k	**2.46**×	n/a
`reopen`	Clean reopen	121 ms	1,369 ms	**11.3**×	yes
`recover`	Open after SIGKILL	128 ms	132 ms	**1.03**×	n/a
`disk`	Disk, after load	91.2 MiB	432.2 MiB	**4.74**×	n/a
`disk.churned`	Disk, after deletes	119.8 MiB	293.3 MiB	**2.45**×	n/a
`disk.compacted`	Disk, after compaction	90.8 MiB	273.5 MiB	**3.01**×	n/a

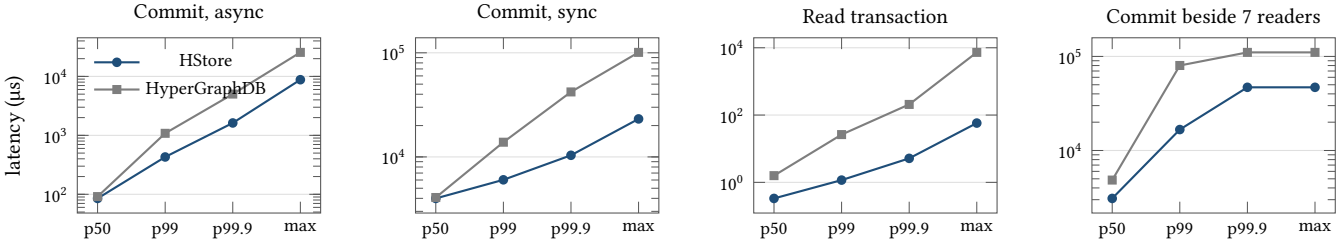


Figure 3: Latency percentiles, median of each percentile over runs (log scale). From left: single-update commits with asynchronous and with synchronous commits, single-lookup read transactions, and commits of the open-loop writer while 7 threads read.

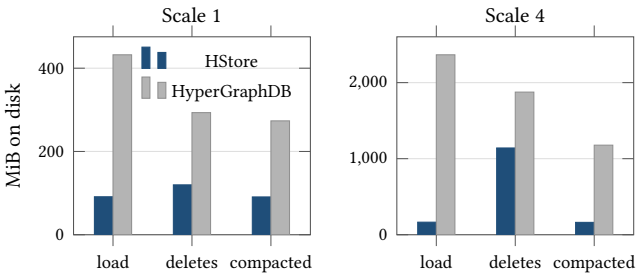


Figure 4: Bytes on disk after load, after deleting 20% of the hyperedges, and after a full space reclamation, median over runs.

4.7 Resources

HStore writes fewer bytes per operation than HyperGraphDB in every write workload of every configuration (fig. 5 and appendix A). A single-update commit writes 649 B in HStore and 9.1 KiB in HyperGraphDB. A batched update writes 221 B against 9.0 KiB. Over a whole run at scale 1, HStore wrote 100.3 MiB and HyperGraphDB 2.96 GiB.

Table 7 summarises memory and reclamation. HStore retained 189.2 MiB of heap after ingest at scale 1, against 118.6 MiB for HyperGraphDB. Per-workload allocation and GC time are in appendix A. At scale 1, HStore allocates less per operation in every read workload and slightly more per inserted node and hyperedge, and it allocates more while recovering. At scale 4 it allocates more per member lookup and per two-hop query, the cache misses discussed in section 4.8.

4.8 Larger data: scale 4

At scale 4 the dataset has 1,453,427 incidences, and the files of both systems are several times larger than their 64 MiB caches. Most of HStore's leads hold or grow. Hyperedge insertion becomes 6.15× faster (11.7 k against 1.9 k per second), batched updates 16.8× and single-update commits 2.58×, the last with overlapping ranges. Co-membership counts are 8.62× faster, incidence lookups 2.64× (ranges overlapping), and cold lookups after a reopen 29.2×.

Two read workloads reverse. HStore enumerates the

Table 6: S1 async: latency per operation, median of each percentile over runs.

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.3 µs	1.2 µs	5.2 µs	58 µs	1.6 µs	26 µs	207 µs	7.29 ms
`latency.commit`	Commit	85 µs	430 µs	1.62 ms	8.79 ms	91 µs	1.08 ms	5.02 ms	25.56 ms
`mixed.read`	Read beside writer	1.6 µs	13 µs	57 µs	51.62 ms	2.9 µs	74 µs	1.03 ms	87.42 ms
`mixed.write`	Commit beside readers	3.09 ms	16.70 ms	46.96 ms	46.96 ms	4.85 ms	80.12 ms	110.43 ms	110.43 ms
`compaction.baseline`	Commit, no compaction	5.06 ms	9.63 ms	10.60 ms	10.60 ms	4.87 ms	27.99 ms	52.86 ms	52.86 ms
`compaction`	Commit during compaction	5.15 ms	9.39 ms	16.03 ms	16.03 ms	5.11 ms	13.32 ms	33.70 ms	33.70 ms

Table 7: Resources per configuration, median over runs. HS: HStore, HG: HyperGraphDB. Space reclaimed is the drop in disk size across the compaction window; it is negative when the writer added more than the reclamation freed.

	S1 async		S1 sync		S4 async	
Measure	HS	HG	HS	HG	HS	HG
Retained heap after ingest	189.2 MiB	118.6 MiB	181.4 MiB	118.4 MiB	424.5 MiB	302.7 MiB
Bytes written, whole run	100.3 MiB	2.96 GiB	104.1 MiB	3.27 GiB	2.06 GiB	92.65 GiB
GC pause, whole run (ms)	238	1,043	386	1,317	4,807	19,782
Reclamation time (ms)	135	50	178	56	77	1,124,686
Space reclaimed	30.7 MiB	12.7 MiB	34.6 MiB	52.1 MiB	−3.1 MiB	709.1 MiB
Acknowledged before kill	25,000	25,000	25,000	25,000	100,000	100,000
Acknowledged lost (max)	0	0	0	0	0	0

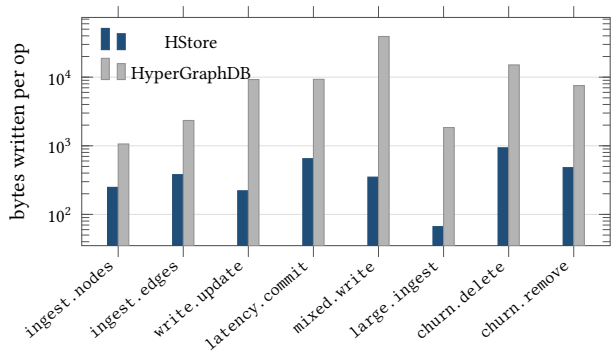


Figure 5: Bytes written per operation, S1 async, median over runs (log scale). Writes include each system's log, data files, checkpoints and cleaning.

members of 218.4 k hyperedges per second against HyperGraphDB's 1.72 M, a ratio of 0.13×. Two-hop neighbourhoods, which enumerate the members of every incident hyperedge, follow at 0.19×. The resource counters show why. At scale 4 HStore's member lookups miss its node cache: each one reads about 778 bytes from the files and allocates 8.8 KiB while decoding the catalog page that holds the edge record, and a two-hop query allocates 130.7 KiB. HyperGraphDB serves the same lookups from its atom cache, which holds each link's targets as a Java array and is not limited by the 64 MiB budget: it allocates 50 B per member lookup. Section 5 returns to this.

On disk, HyperGraphDB's files after loading are 2.31 GiB against HStore's 165.1 MiB. After the deletes, HStore's files grow to 1.11 GiB. The deletes rewrite a large part of the incidence index, and the replaced pages remain reachable from the 64 retained generations until the next restart releases that history. The benchmark restarts the store after the deletes, and during the 10 s `compaction.baseline` window that follows, HStore's background maintenance released and compacted that space. When the measured compaction window began, the explicit compaction took 77 ms and found almost nothing left to reclaim, and the final size was 162.6 MiB. JE's cleaner ran for 1,124,686 ms in the same window and brought HyperGraphDB from 1.83 GiB to 1.15 GiB.

A clean reopen is 5.09× faster in HStore, and opening after a crash takes 203 ms against 545 ms. Over the whole run, HStore wrote 2.06 GiB and HyperGraphDB 92.65 GiB, most of it during cleaning. HStore's single-update commits have a lower p99 (1.34 ms against 3.19 ms) and p99.9, but a higher maximum: 68.55 ms against 9.91 ms.

4.9 Synchronous commits

With synchronous commits, every acknowledged commit has been forced to stable storage. On this machine a forced write costs about 4 ms, and it dominates single-update transactions in both systems. HStore commits 250 of them per second and HyperGraphDB 203 (1.23×). The medians are 3.99 ms and 4.07 ms, one forced write each, and HStore's lead comes from the tail: 10.37 ms against 42.10 ms at p99.9.

Batched work spreads the forced write over 1,000 operations. HStore inserts nodes 2.35× as fast as HyperGraphDB and hyperedges 1.21× as fast (ranges overlapping), and updates values 11.2× as fast. The open-loop writer of `mixed` commits 10 updates every 10 ms. HStore held that schedule, with a median commit latency of 6.28 ms. HyperGraphDB fell behind it: its commits waited 270.86 ms at the median, and its slowest run reached 937 updates per second.

Under synchronous commits, JE reports about 210 KiB written per single-update commit, against about 9 KiB with asynchronous commits. The extra bytes come with the log

flush that the adapter adds (section 2.3). We have not established what JE writes in that flush, so the bytes-written figures for synchronous single commits in appendix A should not be read as a comparison.

Reads do not commit, and HStore leads every read workload in this configuration as well (table 4). Recovery after SIGKILL takes 191 ms in HStore and 139 ms in HyperGraphDB at the median. The ranges overlap, and one HStore run took 1.6 s. As in every other configuration, no run of either system lost an acknowledged commit.

5 Discussion

Incidence and intersection. An incidence lookup in HStore reads one entry of the reverse index. For most atoms the whole incidence set is stored inline in that entry, and for hubs it is the root of a small tree. The visitor API hands each hyperedge id to the caller without allocating, which is why `read.incidence` allocates about one byte per lookup. HyperGraphDB opens a JE cursor on a duplicate set, decodes 16-byte handles, and builds a result object per lookup. With a 64 MiB JE cache, part of each lookup also misses the cache and reads from the log files (appendix A). Co-membership adds a second effect. HStore intersects two sorted incidence trees and skips every subtree whose key range cannot match, so the cost follows the smaller set. HyperGraphDB plans `hg.and` as a general conjunctive query over two result sets. The two-hop workload combines both lookups, so it inherits both advantages.

Member enumeration, and where HyperGraphDB leads. HyperGraphDB keeps a link's targets as a Java array in its atom cache, so enumerating the members of a cached link is an array loop with almost no allocation. HStore reads a hyperedge's members from its record in the catalog tree: from the leaf stored inline in the record for a small edge, or from the edge's own membership tree. At scale 1 the catalog fits in HStore's node cache, and HStore is faster at the median, though the run ranges overlap. At scale 4 it does not. Member lookups then miss the cache and decode catalog pages again, which costs 8.8 KiB of allocation per lookup on average, while HyperGraphDB's atom cache, which the 64 MiB budget does not limit, still holds the links. This is the clearest weakness this study found in HStore. Two directions would address it: smaller catalog pages, or a cache of decoded edge records separate from the node cache. Both trade memory for this access pattern, and neither is in the version measured here.

Small writes. A single-update commit in HStore appends a log record of a few hundred bytes and publishes new roots in memory. The tree pages are written later, by a checkpoint that writes each changed node once for all the commits since the previous one. A JE commit appends to its log a new version of each changed record and of the B+tree internal nodes above it, in every JE database the change touches. The difference shows directly in bytes written per update (fig. 5). Under asynchronous commits, the per-commit cost that remains in HStore is the commit itself: copying the changed tree paths in memory and publishing a generation. That cost is close to JE's at the median, and the gap opens in the tail.

Tail latency. HStore readers pin an immutable generation and never wait for a writer, a checkpoint or a compaction. Writers wait only for the commit lock, which a checkpoint takes just to swap stored references into the head. HyperGraphDB reads run inside JE transactions, which take record locks, and JE's cleaner and checkpointer compete for the same latches and log. In 15 of the 18 latency measurements (six workloads in three configurations), the gap between the two systems is wider at p99 and p99.9 than at the median. The three exceptions are the `mixed` writer under synchronous commits, where HyperGraphDB fell behind its schedule and the gap is largest at the median, and single-update commits and the `mixed` writer at scale 4, where HStore's advantage stays between 1.3 and 2.5 times across percentiles. The single slowest commit at scale 4 was HStore's.

Giant hyperedges. HStore bulk-builds a large membership tree bottom-up in one pass, and a membership probe is one descent of that tree. HyperGraphDB stores the giant link as one record holding 50,000 handles (200,000 at scale 4). A membership probe goes through the node's incidence set instead.

Footprint and reclamation. Several layout choices keep HStore's files small: 8-byte ids stored as varint gaps instead of 16-byte handles, node images packed into 64-byte units, and small member trees and incidence sets stored inline in their parent record. HStore's log is also truncated at each checkpoint. A JE log keeps every superseded version until the cleaner copies the live entries out of a file and deletes it, and at scale 4 cleaning dominated HyperGraphDB's bytes written. HStore's weak point is the period right after a large delete. Pages replaced by the deletes stay on disk while any of the 64 retained generations can reach them, so the files grow first and shrink only after the history moves on, here at the restart that follows. An application that deletes heavily and needs its disk space back quickly would set a smaller `history_limit`.

Opening and recovery. After a clean shutdown, HStore reads one catalog image and finds nothing in the log to replay. HyperGraphDB's open takes longer and allocates far more heap: 1.42 GiB per reopen at scale 1, against 25.7 MiB for HStore. After a crash, HStore replays the logged key changes of every commit since its last checkpoint, and JE replays its log from its last checkpoint. HStore's replay cost grows with the number of commits since the last checkpoint, which the write buffer and the checkpoint thresholds bound. The recovery times of the two systems overlap in our runs, so we do not claim a difference.

What HStore spends. HStore retains more heap after ingest than HyperGraphDB (table 7). The retained heap includes nodes not yet written by a checkpoint, up to the 64 MiB write buffer, and the node cache, whose 64 MiB bud-

get is an estimate that runs below the true heap size. HStore also allocates slightly more per inserted node at scale 1 and pays more GC time during ingest there, and more GC time during the deletes at scale 4. Its worst single commit at scale 4 was slower than HyperGraphDB's, even though its p99 and p99.9 were lower.

6 Threats to validity

One machine, not idle. All numbers come from one laptop that was also running other work. Sampled before each run, the one-minute load average was between 4.4 and 16.7 (median 8.8) on 10 cores. That figure includes the tail of the previous run, whose parallel workloads use 8 threads, so only part of it is outside work. Absolute throughput will differ on a server, and short workloads are sensitive to interference. We report medians, show every run in appendix B, and mark ratios whose run ranges overlap. Conclusions in this report rest only on ratios that are large compared with that spread.

Durability. On this machine one `FileChannel.force` after a small write takes about 3.8 ms at the median, the cost of a write that reaches stable storage. Both systems pay it on every synchronous commit: HStore through its group commit, HyperGraphDB through the explicit log flush described in section 2.3. Without that flush, HyperGraphDB's synchronous configuration would not sync at all. The `recover` workload kills a process, not the machine, so it measures recovery time and the system's own buffering, not power-failure durability. HStore's behaviour under torn writes and lost syncs is covered by its crash-injection tests, which are outside the scope of this report.

Memory fits in RAM. At both scales the files fit in the operating system's page cache. Reads that miss a system's own 64 MiB cache are served from memory, not from the SSD. The cold-read numbers therefore measure decoding and cache admission, not device latency.

Cache budgets. HStore counts its 64 MiB node cache against an estimate of the heap each decoded node takes, which runs 10 to 40% below the true size. JE's 64 MiB budget is exact, but HyperGraphDB's atom and incidence caches are not capped in bytes. Neither side therefore holds exactly 64 MiB. The retained heap in table 7 shows what each kept after ingest.

Workload coverage. The dataset is synthetic. It uses set hyperedges without roles, properties or ordered members, and node values of a few bytes. Queries are single storage operations, not HQL or HyperGraphDB queries, so query planning is not measured. Workloads run in a fixed order within one process, so each one starts from the state the previous one left: cache contents, garbage, and for HStore, unwritten nodes waiting for a checkpoint.

Versions. HyperGraphDB 1.4 is built against Berkeley DB JE 5.0.73, which is older than current JE releases. We did not try other JE versions or tuning beyond the cache size. Both systems otherwise use their default settings.

Authorship. The HStore authors wrote the harness and both adapters. To limit bias, both adapters call the most direct storage API available, every read result is cross-checked between the systems, and both receive the same warm-up and cache budget. The harness, the adapters and the raw results are published with HStore.

7 Conclusion

At the storage level, HStore is faster than HyperGraphDB on the operations that hypergraph workloads repeat most: finding the hyperedges that contain a node, intersecting those sets, and updating values. It writes and stores far fewer bytes doing so. Its design gives a clear reason for each of these results, and the same design explains its main weakness. When the catalog does not fit in the node cache, member enumeration falls well behind HyperGraphDB, whose atom cache keeps links as arrays outside any byte budget. Synchronous commits are bound by the device in both systems. Crash recovery lost no acknowledged commit in either system.

The numbers come from one laptop under load, and section 6 lists what that limits. The harness, the adapters, the raw results and the script that produced every table in this report are published with HStore, so each result can be checked or repeated on other hardware.

References

[1] B. Iordanov. HyperGraphDB: a generalized graph database. In *Web-Age Information Management (WAIM) Workshops*, LNCS 6185, pages 25–36. Springer, 2010.

[2] Oracle. *Berkeley DB Java Edition Architecture*. Oracle white paper, 2006.

[3] T. L. Veldhuizen. Leapfrog triejoin: a simple, worst-case optimal join algorithm. In *Proc. ICDT*, pages 96–106, 2014.

[4] H. Chu. MDB: a memory-mapped database and backend for OpenLDAP. In *Proc. LDAPCon*, 2011.

[5] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, P. Schwarz. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM TODS* 17(1):94–162, 1992.

[6] M. Rosenblum, J. K. Ousterhout. The design and implementation of a log-structured file system. *ACM TOCS* 10(1):26–52, 1992.

A Full results

Each cell is the median over runs. Throughput is in operations per second (k = 10^3, M = 10^6, G = 10^9). Ratios above 1 favour HStore. "Agree" is *yes* when both systems returned the same checksum in every run.

A.1 Scale 1, asynchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	45.1 k	24.9 k	**1.81**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	28.2 k	22.5 k	**1.25**×	yes
`write.update`	Update node values (1,000 per txn)	78.1 k	5.1 k	**15.3**×	yes
`latency.commit`	Single-update transactions	7.9 k	5.8 k	**1.36**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	16.3 k	3.0 k	**5.46**×	yes
`churn.remove`	Remove one member (1,000 per txn)	24.4 k	8.1 k	**3.00**×	yes
`large.ingest`	Build one giant hyperedge	439.6 k	35.7 k	**12.3**×	yes
`read.incidence`	Incidence set of a node	7.47 M	739.5 k	**10.1**×	yes
`read.members`	Members of a hyperedge	3.69 M	1.70 M	**2.17**×	yes
`read.comembership`	Co-membership count of a pair	447.8 k	32.5 k	**13.8**×	yes
`read.twohop`	Two-hop neighbourhood	301.2 k	74.6 k	**4.04**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	24.44 M	1.69 M	**14.4**×	yes
`latency.read`	One lookup per read txn	2.81 M	269.4 k	**10.4**×	yes
`read.incidence.cold`	Incidence sets after reopen	2.76 M	41.9 k	**65.9**×	yes
`read.incidence.churned`	Incidence sets after deletes	6.89 M	1.11 M	**6.22**×	yes
`large.scan`	Scan the giant hyperedge	2.26 G	610.67 M	**3.70**×	yes
`large.probe`	Membership probe, giant hyperedge	15.62 M	771.9 k	**20.2**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	1.87 M	762.0 k	**2.46**×	n/a
`reopen`	Clean reopen	121 ms	1,369 ms	**11.3**×	yes
`recover`	Open after SIGKILL	128 ms	132 ms	**1.03**×	n/a
`disk`	Disk, after load	91.2 MiB	432.2 MiB	**4.74**×	n/a
`disk.churned`	Disk, after deletes	119.8 MiB	293.3 MiB	**2.45**×	n/a
`disk.compacted`	Disk, after compaction	90.8 MiB	273.5 MiB	**3.01**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.3 µs	1.2 µs	5.2 µs	58 µs	1.6 µs	26 µs	207 µs	7.29 ms
`latency.commit`	Commit	85 µs	430 µs	1.62 ms	8.79 ms	91 µs	1.08 ms	5.02 ms	25.56 ms
`mixed.read`	Read beside writer	1.6 µs	13 µs	57 µs	51.62 ms	2.9 µs	74 µs	1.03 ms	87.42 ms
`mixed.write`	Commit beside readers	3.09 ms	16.70 ms	46.96 ms	46.96 ms	4.85 ms	80.12 ms	110.43 ms	110.43 ms
`compaction.baseline`	Commit, no compaction	5.06 ms	9.63 ms	10.60 ms	10.60 ms	4.87 ms	27.99 ms	52.86 ms	52.86 ms
`compaction`	Commit during compaction	5.15 ms	9.39 ms	16.03 ms	16.03 ms	5.11 ms	13.32 ms	33.70 ms	33.70 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	248 B	1.0 KiB	14.0×	59.8×	**4.28**×
`ingest.edges`	380 B	2.3 KiB	10.3×	63.1×	**6.15**×
`write.update`	221 B	9.0 KiB	6.9×	288.4×	**41.8**×
`latency.commit`	649 B	9.1 KiB	20.3×	290.4×	**14.3**×
`mixed.write`	350 B	38.2 KiB	10.9×	1223.6×	**112**×
`large.ingest`	66 B	1.8 KiB	8.3×	230.6×	**27.8**×
`churn.delete`	935 B	14.7 KiB	25.3×	407.9×	**16.1**×
`churn.remove`	480 B	7.4 KiB	30.0×	471.1×	**15.7**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	19.3 KiB	18.1 KiB	13	0
`ingest.edges`	36.2 KiB	29.1 KiB	97	58
`write.update`	21.1 KiB	196.9 KiB	0	62
`latency.commit`	76.1 KiB	192.7 KiB	5	9
`churn.delete`	50.8 KiB	382.6 KiB	47	106
`churn.remove`	26.6 KiB	168.1 KiB	69	35
`large.ingest`	2.0 KiB	31.6 KiB	0	26
`read.incidence`	1 B	99 B	0	0
`read.members`	40 B	49 B	0	0
`read.comembership`	3.9 KiB	7.7 KiB	0	0
`read.twohop`	373 B	1.8 KiB	0	0
`read.incidence.parallel`	2 B	643 B	0	0
`latency.read`	269 B	4.4 KiB	0	0
`read.incidence.cold`	266 B	6.5 KiB	0	70
`read.incidence.churned`	1 B	98 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	0 B	101 B	0	0
`mixed.read`	112 B	2.3 KiB	38	225
`reopen`	25.7 MiB	1.42 GiB	0	10
`recover`	115.5 MiB	78.1 MiB	0	0

A.2 Scale 1, synchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	29.7 k	12.6 k	**2.35**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	19.9 k	16.4 k	**1.21**×	yes
`write.update`	Update node values (1,000 per txn)	40.3 k	3.6 k	**11.2**×	yes
`latency.commit`	Single-update transactions	250	203	**1.23**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	10.5 k	2.3 k	**4.53**×	yes
`churn.remove`	Remove one member (1,000 per txn)	21.7 k	6.4 k	**3.40**×	yes
`large.ingest`	Build one giant hyperedge	264.8 k	28.9 k	**9.16**×	yes
`read.incidence`	Incidence set of a node	3.66 M	384.6 k	**9.52**×	yes
`read.members`	Members of a hyperedge	2.55 M	1.18 M	**2.17**×	yes
`read.comembership`	Co-membership count of a pair	269.0 k	9.3 k	**29.0**×	yes
`read.twohop`	Two-hop neighbourhood	174.4 k	38.2 k	**4.57**×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	7.89 M	942.1 k	**8.37**×	yes
`latency.read`	One lookup per read txn	1.81 M	222.2 k	**8.13**×	yes
`read.incidence.cold`	Incidence sets after reopen	1.92 M	35.7 k	**53.7**×	yes
`read.incidence.churned`	Incidence sets after deletes	4.18 M	832.4 k	**5.02**×	yes
`large.scan`	Scan the giant hyperedge	1.79 G	401.18 M	**4.47**×	yes
`large.probe`	Membership probe, giant hyperedge	9.60 M	867.4 k	**11.1**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	1.58 M	623.7 k	**2.54**×	n/a
`reopen`	Clean reopen	165 ms	1,274 ms	**7.73**×	yes
`recover`	Open after SIGKILL	191 ms	139 ms	0.73×	n/a
`disk`	Disk, after load	91.2 MiB	350.9 MiB	**3.85**×	n/a
`disk.churned`	Disk, after deletes	123.6 MiB	278.1 MiB	**2.25**×	n/a
`disk.compacted`	Disk, after compaction	90.8 MiB	235.4 MiB	**2.59**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.5 µs	1.1 µs	3.0 µs	50 µs	2.0 µs	30 µs	203 µs	10.25 ms
`latency.commit`	Commit	3.99 ms	6.03 ms	10.37 ms	23.18 ms	4.07 ms	13.84 ms	42.10 ms	100.95 ms
`mixed.read`	Read beside writer	1.9 µs	19 µs	131 µs	65.11 ms	3.3 µs	82 µs	1.62 ms	67.14 ms
`mixed.write`	Commit beside readers	6.28 ms	185.14 ms	200.32 ms	200.32 ms	270.86 ms	763.31 ms	769.14 ms	769.14 ms
`compaction.baseline`	Commit, no compaction	6.59 ms	9.05 ms	12.79 ms	12.79 ms	6.57 ms	24.93 ms	56.98 ms	56.98 ms
`compaction`	Commit during compaction	6.65 ms	10.01 ms	12.61 ms	12.61 ms	6.66 ms	56.13 ms	89.15 ms	89.15 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	248 B	1.0 KiB	14.0×	59.7×	**4.28**×
`ingest.edges`	380 B	2.2 KiB	10.3×	61.6×	**6.00**×
`write.update`	221 B	10.6 KiB	6.9×	339.9×	**49.2**×
`latency.commit`	649 B	210.2 KiB	20.3×	6724.9×	**332**×
`mixed.write`	350 B	28.7 KiB	10.9×	919.4×	**84.2**×
`large.ingest`	66 B	1.5 KiB	8.3×	193.2×	**23.3**×
`churn.delete`	950 B	11.8 KiB	25.7×	328.2×	**12.8**×
`churn.remove`	668 B	7.6 KiB	41.7×	483.9×	**11.6**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	19.2 KiB	18.0 KiB	11	0
`ingest.edges`	36.0 KiB	27.8 KiB	136	118
`write.update`	20.7 KiB	244.2 KiB	0	56
`latency.commit`	128.7 KiB	4.2 MiB	15	268
`churn.delete`	54.1 KiB	286.5 KiB	63	219
`churn.remove`	29.7 KiB	164.7 KiB	32	47
`large.ingest`	2.0 KiB	26.2 KiB	0	36
`read.incidence`	1 B	102 B	0	0
`read.members`	40 B	49 B	0	0
`read.comembership`	3.9 KiB	8.1 KiB	0	62
`read.twohop`	373 B	1.8 KiB	0	0
`read.incidence.parallel`	39 B	545 B	0	0
`latency.read`	269 B	4.2 KiB	0	0
`read.incidence.cold`	266 B	6.3 KiB	0	72
`read.incidence.churned`	1 B	98 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	0 B	98 B	0	0
`mixed.read`	127 B	2.1 KiB	76	162
`reopen`	25.7 MiB	1.19 GiB	0	17
`recover`	114.8 MiB	78.4 MiB	0	0

A.3 Scale 4, asynchronous commits

Workload	Operation	HStore	HyperGraphDB	Ratio	Agree
`ingest.nodes`	Insert nodes (1,000 per txn)	27.1 k	21.4 k	**1.27**×	yes
`ingest.edges`	Insert hyperedges (1,000 per txn)	11.7 k	1.9 k	**6.15**×	yes
`write.update`	Update node values (1,000 per txn)	20.8 k	1.2 k	**16.8**×	yes
`latency.commit`	Single-update transactions	5.2 k	2.0 k	**2.58**×	yes
`churn.delete`	Delete hyperedges (1,000 per txn)	2.1 k	1.5 k	**1.41**×	yes
`churn.remove`	Remove one member (1,000 per txn)	2.8 k	2.7 k	**1.05**×	yes
`large.ingest`	Build one giant hyperedge	212.7 k	12.6 k	**16.8**×	yes
`read.incidence`	Incidence set of a node	2.66 M	1.01 M	**2.64**×	yes
`read.members`	Members of a hyperedge	218.4 k	1.72 M	0.13×	yes
`read.comembership`	Co-membership count of a pair	390.3 k	45.3 k	**8.62**×	yes
`read.twohop`	Two-hop neighbourhood	15.0 k	78.1 k	0.19×	yes
`read.incidence.parallel`	Incidence sets, 8 threads	5.18 M	1.78 M	**2.90**×	yes
`latency.read`	One lookup per read txn	1.15 M	364.1 k	**3.16**×	yes
`read.incidence.cold`	Incidence sets after reopen	346.0 k	11.8 k	**29.2**×	yes
`read.incidence.churned`	Incidence sets after deletes	3.45 M	777.1 k	**4.44**×	yes
`large.scan`	Scan the giant hyperedge	1.60 G	564.34 M	**2.83**×	yes
`large.probe`	Membership probe, giant hyperedge	6.82 M	991.1 k	**6.88**×	yes
`mixed.read`	Reads beside a 1,000 updates/s writer	1.76 M	883.7 k	**1.99**×	n/a
`reopen`	Clean reopen	238 ms	1,210 ms	**5.09**×	yes
`recover`	Open after SIGKILL	203 ms	545 ms	**2.69**×	n/a
`disk`	Disk, after load	165.1 MiB	2.31 GiB	**14.3**×	n/a
`disk.churned`	Disk, after deletes	1.11 GiB	1.83 GiB	**1.64**×	n/a
`disk.compacted`	Disk, after compaction	162.6 MiB	1.15 GiB	**7.24**×	n/a

Latency (median of each percentile over runs)

		HStore				HyperGraphDB			
Workload	Operation	p50	p99	p99.9	max	p50	p99	p99.9	max
`latency.read`	Read transaction	0.7 µs	1.4 µs	6.7 µs	99 µs	1.4 µs	21 µs	54 µs	7.86 ms
`latency.commit`	Commit	118 µs	1.34 ms	3.96 ms	68.55 ms	297 µs	3.19 ms	7.89 ms	9.91 ms
`mixed.read`	Read beside writer	1.8 µs	12 µs	114 µs	76.02 ms	2.4 µs	54 µs	1.04 ms	96.66 ms
`mixed.write`	Commit beside readers	3.91 ms	52.77 ms	78.00 ms	78.00 ms	5.95 ms	74.80 ms	101.47 ms	101.47 ms
`compaction.baseline`	Commit, no compaction	5.38 ms	10.04 ms	11.07 ms	11.07 ms	5.54 ms	36.70 ms	114.47 ms	114.47 ms
`compaction`	Commit during compaction	4.89 ms	9.95 ms	13.56 ms	13.56 ms	5.62 ms	92.77 ms	210.77 ms	429.52 ms

	Bytes written per operation		Write amplification		
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB	Ratio
`ingest.nodes`	330 B	1.5 KiB	17.9×	85.2×	**4.76**×
`ingest.edges`	1.2 KiB	28.4 KiB	32.2×	783.5×	**24.4**×
`write.update`	498 B	38.9 KiB	15.5×	1245.6×	**80.1**×
`latency.commit`	1.7 KiB	28.3 KiB	54.3×	904.1×	**16.7**×
`mixed.write`	5.3 KiB	47.7 KiB	170.5×	1527.9×	**8.96**×
`large.ingest`	71 B	6.1 KiB	8.9×	780.7×	**88.0**×
`churn.delete`	15.6 KiB	42.0 KiB	429.4×	1157.4×	**2.70**×
`churn.remove`	13.1 KiB	22.4 KiB	837.2×	1433.6×	**1.71**×

	Heap allocated per operation		GC pause (ms)	
Workload	HStore	HyperGraphDB	HStore	HyperGraphDB
`ingest.nodes`	25.8 KiB	25.6 KiB	181	163
`ingest.edges`	113.8 KiB	876.0 KiB	1813	2567
`write.update`	50.2 KiB	1.2 MiB	47	313
`latency.commit`	157.2 KiB	860.4 KiB	68	17
`churn.delete`	321.3 KiB	1.4 MiB	1640	893
`churn.remove`	153.5 KiB	738.4 KiB	289	103
`large.ingest`	2.7 KiB	172.3 KiB	0	167
`read.incidence`	2 B	98 B	0	8
`read.members`	8.8 KiB	50 B	0	0
`read.comembership`	4.0 KiB	8.1 KiB	0	0
`read.twohop`	130.7 KiB	1.8 KiB	7	0
`read.incidence.parallel`	91 B	1.1 KiB	0	0
`latency.read`	293 B	6.2 KiB	0	8
`read.incidence.cold`	1024 B	24.1 KiB	0	130
`read.incidence.churned`	1 B	99 B	0	0
`large.scan`	0 B	0 B	0	0
`large.probe`	0 B	98 B	0	0
`mixed.read`	157 B	2.9 KiB	103	213
`reopen`	63.2 MiB	2.28 GiB	0	19
`recover`	221.2 MiB	323.1 MiB	0	0

B Run-to-run variability

The tables below give the minimum and maximum over runs and the coefficient of variation (standard deviation over mean) of every throughput workload, followed by the value of every run.

B.1 Scale 1, asynchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
`ingest.nodes`	32.9 k–66.1 k	23%	11.2 k–33.0 k	38%
`ingest.edges`	22.5 k–37.3 k	21%	11.1 k–36.0 k	35%
`write.update`	53.2 k–82.9 k	18%	2.9 k–8.7 k	38%
`latency.commit`	5.3 k–13.6 k	36%	4.6 k–9.6 k	27%
`churn.delete`	6.1 k–20.5 k	39%	1.6 k–6.4 k	47%
`churn.remove`	14.5 k–33.2 k	25%	5.7 k–15.6 k	36%
`large.ingest`	261.9 k–480.9 k	19%	13.8 k–54.1 k	42%
`read.incidence`	4.00 M–7.63 M	21%	221.9 k–927.8 k	37%
`read.members`	1.53 M–5.20 M	36%	482.8 k–2.22 M	52%
`read.comembership`	127.2 k–651.9 k	41%	18.5 k–54.8 k	35%
`read.twohop`	240.8 k–347.6 k	13%	18.6 k–130.6 k	55%
`read.incidence.parallel`	17.13 M–34.80 M	23%	637.3 k–2.18 M	34%
`latency.read`	1.97 M–3.46 M	21%	106.9 k–337.6 k	31%
`read.incidence.cold`	278.0 k–3.03 M	44%	15.5 k–55.2 k	47%
`read.incidence.churned`	2.63 M–7.49 M	38%	785.1 k–2.40 M	43%
`large.scan`	1.70 G–3.73 G	33%	308.74 M–1.17 G	49%
`large.probe`	2.54 M–19.29 M	42%	252.0 k–1.07 M	45%
`mixed.read`	1.78 M–2.53 M	14%	316.8 k–1.02 M	39%

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
`ingest.nodes`	HStore	45.0 k	66.1 k	47.4 k	45.1 k	32.9 k
	HyperGraphDB	33.0 k	24.9 k	11.2 k	32.1 k	14.8 k
`ingest.edges`	HStore	34.1 k	37.3 k	22.6 k	22.5 k	28.2 k
	HyperGraphDB	36.0 k	26.3 k	11.1 k	20.0 k	22.5 k
`write.update`	HStore	80.4 k	53.2 k	56.0 k	82.9 k	78.1 k
	HyperGraphDB	8.7 k	5.1 k	2.9 k	6.5 k	3.7 k
`latency.commit`	HStore	13.6 k	7.9 k	5.3 k	10.5 k	5.8 k
	HyperGraphDB	9.6 k	5.8 k	4.6 k	5.4 k	6.8 k
`churn.delete`	HStore	20.5 k	20.5 k	6.1 k	10.1 k	16.3 k
	HyperGraphDB	6.4 k	3.0 k	1.6 k	2.9 k	3.0 k
`churn.remove`	HStore	23.7 k	33.2 k	14.5 k	26.7 k	24.4 k
	HyperGraphDB	15.6 k	8.1 k	5.7 k	9.0 k	8.1 k
`large.ingest`	HStore	405.7 k	439.6 k	261.9 k	453.1 k	480.9 k
	HyperGraphDB	54.0 k	27.8 k	13.8 k	35.7 k	54.1 k
`read.incidence`	HStore	7.63 M	7.47 M	4.00 M	7.63 M	6.09 M
	HyperGraphDB	739.5 k	916.5 k	221.9 k	927.8 k	706.4 k
`read.members`	HStore	2.79 M	5.20 M	4.09 M	3.69 M	1.53 M
	HyperGraphDB	2.17 M	1.70 M	482.8 k	2.22 M	620.4 k
`read.comembership`	HStore	651.9 k	577.9 k	127.2 k	397.5 k	447.8 k
	HyperGraphDB	54.8 k	45.8 k	29.7 k	18.5 k	32.5 k
`read.twohop`	HStore	280.8 k	240.8 k	301.2 k	339.4 k	347.6 k
	HyperGraphDB	130.6 k	74.6 k	18.6 k	43.5 k	76.5 k
`read.incidence.parallel`	HStore	17.13 M	22.66 M	24.44 M	34.80 M	26.41 M
	HyperGraphDB	2.18 M	1.83 M	637.3 k	1.38 M	1.69 M
`latency.read`	HStore	3.46 M	2.19 M	1.97 M	3.15 M	2.81 M
	HyperGraphDB	298.4 k	337.6 k	106.9 k	269.4 k	261.6 k
`read.incidence.cold`	HStore	2.75 M	2.76 M	278.0 k	2.80 M	3.03 M
	HyperGraphDB	55.2 k	15.5 k	17.1 k	54.0 k	41.9 k
`read.incidence.churned`	HStore	6.89 M	2.63 M	3.33 M	7.49 M	7.31 M
	HyperGraphDB	2.40 M	1.11 M	1.17 M	785.1 k	1.08 M

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
large.scan	HStore	3.73 G	3.66 G	1.92 G	2.26 G	1.70 G
	HyperGraphDB	1.17 G	308.74 M	351.89 M	610.67 M	729.55 M
large.probe	HStore	19.29 M	15.49 M	2.54 M	16.43 M	15.62 M
	HyperGraphDB	771.9 k	804.7 k	396.2 k	1.07 M	252.0 k
mixed.read	HStore	2.53 M	2.24 M	1.87 M	1.86 M	1.78 M
	HyperGraphDB	1.02 M	431.1 k	316.8 k	791.9 k	762.0 k
reopen	HStore	116 ms	119 ms	373 ms	138 ms	121 ms
	HyperGraphDB	1369 ms	1491 ms	2088 ms	1111 ms	940 ms
recover	HStore	128 ms	163 ms	356 ms	112 ms	107 ms
	HyperGraphDB	82 ms	143 ms	105 ms	157 ms	132 ms
disk	HStore	91.2 MiB	91.2 MiB	93.2 MiB	91.2 MiB	91.1 MiB
	HyperGraphDB	374.2 MiB	432.2 MiB	535.2 MiB	409.8 MiB	442.8 MiB
disk.churned	HStore	119.3 MiB	115.9 MiB	129.7 MiB	124.1 MiB	119.8 MiB
	HyperGraphDB	271.7 MiB	348.9 MiB	261.2 MiB	293.3 MiB	337.2 MiB
disk.compacted	HStore	90.8 MiB	90.8 MiB	90.8 MiB	90.8 MiB	90.8 MiB
	HyperGraphDB	204.0 MiB	290.7 MiB	288.6 MiB	273.5 MiB	257.8 MiB

B.2 Scale 1, synchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
ingest.nodes	22.2 k–31.5 k	12%	9.3 k–15.6 k	18%
ingest.edges	5.1 k–25.2 k	39%	11.3 k–19.4 k	16%
write.update	6.0 k–50.8 k	44%	2.7 k–4.6 k	19%
latency.commit	238–255	2%	160–216	10%
churn.delete	7.7 k–14.8 k	24%	2.0 k–3.5 k	23%
churn.remove	7.6 k–26.6 k	34%	4.6 k–8.9 k	23%
large.ingest	100.1 k–348.7 k	34%	18.6 k–59.3 k	43%
read.incidence	722.8 k–4.40 M	43%	202.5 k–542.0 k	30%
read.members	1.28 M–2.71 M	31%	605.2 k–1.61 M	39%
read.comembership	73.1 k–359.8 k	40%	6.6 k–22.4 k	51%
read.twohop	112.2 k–239.4 k	26%	14.8 k–62.3 k	40%
read.incidence.parallel	3.87 M–17.03 M	48%	626.7 k–1.70 M	36%
latency.read	601.8 k–2.27 M	34%	170.4 k–251.1 k	16%
read.incidence.cold	1.50 M–2.10 M	11%	19.6 k–47.1 k	31%
read.incidence.churned	3.91 M–4.73 M	7%	239.5 k–1.28 M	52%
large.scan	1.36 G–2.53 G	23%	209.07 M–495.72 M	30%
large.probe	7.71 M–10.33 M	9%	123.2 k–1.19 M	68%
mixed.read	1.25 M–1.63 M	9%	528.8 k–731.2 k	12%

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
ingest.nodes	HStore	29.7 k	22.2 k	31.1 k	27.5 k	31.5 k
	HyperGraphDB	12.1 k	9.3 k	15.6 k	15.6 k	12.6 k
ingest.edges	HStore	23.1 k	5.1 k	25.2 k	16.8 k	19.9 k
	HyperGraphDB	11.3 k	16.9 k	16.4 k	19.4 k	15.5 k
write.update	HStore	42.7 k	6.0 k	33.9 k	40.3 k	50.8 k
	HyperGraphDB	3.2 k	4.5 k	3.6 k	4.6 k	2.7 k
latency.commit	HStore	238	248	250	253	255
	HyperGraphDB	203	160	212	216	188
churn.delete	HStore	7.7 k	14.8 k	14.7 k	10.5 k	9.9 k
	HyperGraphDB	3.4 k	2.0 k	3.5 k	2.2 k	2.3 k
churn.remove	HStore	21.7 k	7.6 k	16.8 k	22.5 k	26.6 k
	HyperGraphDB	6.4 k	4.6 k	8.9 k	5.6 k	8.0 k
large.ingest	HStore	100.1 k	346.9 k	264.8 k	254.5 k	348.7 k
	HyperGraphDB	18.6 k	36.9 k	59.3 k	28.9 k	23.1 k
read.incidence	HStore	4.37 M	722.8 k	2.81 M	3.66 M	4.40 M
	HyperGraphDB	202.5 k	324.0 k	384.6 k	542.0 k	399.7 k

Workload	Engine	Run 1	Run 2	Run 3	Run 4	Run 5
`read.members`	HStore	2.68 M	1.38 M	2.71 M	2.55 M	1.28 M
	HyperGraphDB	1.57 M	614.5 k	1.18 M	1.61 M	605.2 k
`read.comembership`	HStore	359.8 k	73.1 k	237.8 k	269.0 k	338.4 k
	HyperGraphDB	9.3 k	7.1 k	22.4 k	6.6 k	11.0 k
`read.twohop`	HStore	142.4 k	112.2 k	239.4 k	205.1 k	174.4 k
	HyperGraphDB	62.3 k	14.8 k	39.6 k	33.3 k	38.2 k
`read.incidence.parallel`	HStore	9.01 M	3.87 M	17.03 M	7.89 M	7.75 M
	HyperGraphDB	1.01 M	626.7 k	1.70 M	800.8 k	942.1 k
`latency.read`	HStore	2.01 M	601.8 k	1.81 M	1.79 M	2.27 M
	HyperGraphDB	250.3 k	170.4 k	178.8 k	251.1 k	222.2 k
`read.incidence.cold`	HStore	1.92 M	1.50 M	1.92 M	2.10 M	2.07 M
	HyperGraphDB	22.1 k	35.7 k	47.1 k	36.5 k	19.6 k
`read.incidence.churned`	HStore	4.02 M	4.54 M	3.91 M	4.73 M	4.18 M
	HyperGraphDB	239.5 k	918.4 k	832.4 k	1.28 M	371.2 k
`large.scan`	HStore	1.79 G	1.51 G	2.53 G	2.09 G	1.36 G
	HyperGraphDB	495.72 M	435.96 M	401.18 M	257.69 M	209.07 M
`large.probe`	HStore	9.04 M	7.71 M	9.60 M	9.63 M	10.33 M
	HyperGraphDB	867.4 k	135.9 k	123.2 k	1.19 M	868.7 k
`mixed.read`	HStore	1.59 M	1.25 M	1.58 M	1.63 M	1.51 M
	HyperGraphDB	667.5 k	551.4 k	731.2 k	623.7 k	528.8 k
`reopen`	HStore	165 ms	166 ms	155 ms	146 ms	165 ms
	HyperGraphDB	1408 ms	1016 ms	1274 ms	1042 ms	2417 ms
`recover`	HStore	1646 ms	156 ms	212 ms	157 ms	191 ms
	HyperGraphDB	139 ms	123 ms	149 ms	171 ms	128 ms
`disk`	HStore	90.7 MiB	92.4 MiB	91.2 MiB	92.7 MiB	90.7 MiB
	HyperGraphDB	341.1 MiB	429.2 MiB	348.2 MiB	350.9 MiB	421.5 MiB
`disk.churned`	HStore	127.4 MiB	128.4 MiB	123.6 MiB	121.6 MiB	123.6 MiB
	HyperGraphDB	271.3 MiB	300.1 MiB	259.0 MiB	278.1 MiB	359.5 MiB
`disk.compacted`	HStore	90.8 MiB	90.8 MiB	90.8 MiB	90.8 MiB	90.8 MiB
	HyperGraphDB	249.7 MiB	233.2 MiB	268.1 MiB	230.3 MiB	235.4 MiB

B.3 Scale 4, asynchronous commits

	HStore		HyperGraphDB	
Workload	min–max	CV	min–max	CV
`ingest.nodes`	23.1 k–54.4 k	40%	18.1 k–39.8 k	36%
`ingest.edges`	8.4 k–29.1 k	55%	1.4 k–2.9 k	30%
`write.update`	11.7 k–64.9 k	72%	1.0 k–2.5 k	40%
`latency.commit`	1.5 k–12.2 k	70%	1.4 k–2.5 k	24%
`churn.delete`	1.5 k–5.0 k	53%	885–1.9 k	28%
`churn.remove`	2.4 k–7.6 k	54%	1.7 k–3.6 k	28%
`large.ingest`	148.7 k–632.0 k	65%	7.6 k–14.0 k	24%
`read.incidence`	721.4 k–3.51 M	51%	466.2 k–1.02 M	31%
`read.members`	164.4 k–441.8 k	44%	1.09 M–2.72 M	36%
`read.comembership`	245.7 k–685.4 k	42%	2.5 k–65.0 k	69%
`read.twohop`	8.9 k–22.5 k	36%	21.4 k–98.7 k	49%
`read.incidence.parallel`	4.05 M–24.38 M	83%	1.04 M–1.86 M	23%
`latency.read`	1.03 M–2.79 M	49%	120.2 k–368.7 k	41%
`read.incidence.cold`	286.7 k–1.83 M	87%	6.0 k–15.3 k	34%
`read.incidence.churned`	2.87 M–6.64 M	38%	767.7 k–1.15 M	20%
`large.scan`	933.83 M–2.83 G	44%	89.03 M–642.99 M	57%
`large.probe`	2.35 M–17.02 M	70%	952.6 k–1.14 M	8%
`mixed.read`	1.35 M–2.10 M	18%	535.4 k–1.02 M	25%

Workload	Engine	Run 1	Run 2	Run 3
`ingest.nodes`	HStore	23.1 k	27.1 k	54.4 k
	HyperGraphDB	18.1 k	21.4 k	39.8 k

Workload	Engine	Run 1	Run 2	Run 3
`ingest.edges`	HStore	8.4 k	11.7 k	29.1 k
	HyperGraphDB	1.4 k	1.9 k	2.9 k
`write.update`	HStore	11.7 k	20.8 k	64.9 k
	HyperGraphDB	1.0 k	1.2 k	2.5 k
`latency.commit`	HStore	1.5 k	5.2 k	12.2 k
	HyperGraphDB	1.4 k	2.0 k	2.5 k
`churn.delete`	HStore	2.1 k	1.5 k	5.0 k
	HyperGraphDB	885	1.5 k	1.9 k
`churn.remove`	HStore	2.4 k	2.8 k	7.6 k
	HyperGraphDB	1.7 k	2.7 k	3.6 k
`large.ingest`	HStore	148.7 k	212.7 k	632.0 k
	HyperGraphDB	7.6 k	12.6 k	14.0 k
`read.incidence`	HStore	721.4 k	2.66 M	3.51 M
	HyperGraphDB	466.2 k	1.02 M	1.01 M
`read.members`	HStore	164.4 k	218.4 k	441.8 k
	HyperGraphDB	1.72 M	1.09 M	2.72 M
`read.comembership`	HStore	390.3 k	245.7 k	685.4 k
	HyperGraphDB	2.5 k	45.3 k	65.0 k
`read.twohop`	HStore	15.0 k	8.9 k	22.5 k
	HyperGraphDB	21.4 k	78.1 k	98.7 k
`read.incidence.parallel`	HStore	5.18 M	4.05 M	24.38 M
	HyperGraphDB	1.04 M	1.78 M	1.86 M
`latency.read`	HStore	1.15 M	1.03 M	2.79 M
	HyperGraphDB	120.2 k	364.1 k	368.7 k
`read.incidence.cold`	HStore	286.7 k	346.0 k	1.83 M
	HyperGraphDB	6.0 k	11.8 k	15.3 k
`read.incidence.churned`	HStore	3.45 M	2.87 M	6.64 M
	HyperGraphDB	767.7 k	777.1 k	1.15 M
`large.scan`	HStore	933.83 M	1.60 G	2.83 G
	HyperGraphDB	89.03 M	564.34 M	642.99 M
`large.probe`	HStore	2.35 M	6.82 M	17.02 M
	HyperGraphDB	952.6 k	991.1 k	1.14 M
`mixed.read`	HStore	1.35 M	1.76 M	2.10 M
	HyperGraphDB	535.4 k	883.7 k	1.02 M
`reopen`	HStore	313 ms	238 ms	127 ms
	HyperGraphDB	1569 ms	1210 ms	914 ms
`recover`	HStore	403 ms	203 ms	177 ms
	HyperGraphDB	1214 ms	362 ms	545 ms
`disk`	HStore	165.2 MiB	165.1 MiB	163.2 MiB
	HyperGraphDB	2.31 GiB	2.32 GiB	2.18 GiB
`disk.churned`	HStore	1.11 GiB	1.11 GiB	1.12 GiB
	HyperGraphDB	1.83 GiB	1.83 GiB	1.90 GiB
`disk.compacted`	HStore	162.6 MiB	162.7 MiB	162.6 MiB
	HyperGraphDB	1.15 GiB	1.19 GiB	1.14 GiB

C Reproducing the results

The harness is the `benchmarks` module of the HStore repository. HyperGraphDB and JE are built from source at the commits named in table 1.

```
benchmarks/install-hypergraphdb.sh      # builds HyperGraphDB 99485a1 and JE 5.0.73
SCALE=1 DURABILITY=async CACHE_MB=64 RUNS=5 THREADS=8 benchmarks/run.sh
SCALE=1 DURABILITY=sync  CACHE_MB=64 RUNS=5 THREADS=8 benchmarks/run.sh
SCALE=4 DURABILITY=async CACHE_MB=64 RUNS=3 THREADS=8 benchmarks/run.sh
```

`run.sh` builds the module, then runs each system `RUNS` times in alternation, one JVM per run, with these flags:

```
java -Xms4g -Xmx4g -XX:+UseParallelGC \
     --add-opens java.base/java.lang=ALL-UNNAMED \
     --add-opens java.base/java.lang.ref=ALL-UNNAMED \
     --add-opens java.base/java.util=ALL-UNNAMED \
     io.hstore.bench.Comparison run --store <hstore|hypergraphdb> \
     --scale <s> --durability <async|sync> --threads 8 --history 64 --cache-mb 64
```

Each run writes one JSON file with every measurement in this report. `Comparison report` turns a set of them into Markdown tables. The tables, figures and quoted numbers in this report were generated from the same files by `benchmarks/report/gen.py`, without manual transcription. The dataset statistics in table 2 are printed by `io.hstore.bench.DatasetStats`, run with the same classpath. The raw files are in `benchmarks/report/results`, and the report is rebuilt with:

```
cd benchmarks/report
python3 gen.py results gen
tectonic report.tex
```

D HStore settings

Setting	Meaning	Value
`page_size`	tree node page size	16 KiB
segment size	pages per segment file	16,384 (256 MiB)
`node_cache_mb`	decoded node cache	64 MiB (set)
`write_buffer_mb`	cap on unwritten tree nodes	64 MiB
`checkpoint_wal_mb`	log or page bytes that trigger a checkpoint	256 MiB
`compaction_live_ratio`	compact segments below this live fraction	0.5
`history_limit`	generations kept for time travel	64
`wal_mode`	log page references, not page images	references
maintenance interval	background checkpoint and compaction pass	500 ms